Simulation-Based Fault Coverage Analysis and Low-Power Optimization of a BIST-Enabled 16-Bit Multiply-Accumulate Unit

Bonthala Saikiran
SENSE
Vellore Institute of Technology, Chennai
bonthala.saikiran2024@vitstudent.ac.in

Allen Joe A
SENSE
Vellore Institute of Technology, Chennai
allenjoe.a2024@vitstudent.ac.in

Papanasam E
SENSE
Vellore Institute of Technology, Chennai
papanasam.e@vit.ac.in

***Abstract*—This paper presents the design, verification, and low-power optimization of a built-in self-test (BIST) architecture for a 16-bit unsigned multiply-accumulate (MAC) unit. The BIST core comprises two 16-bit linear feedback shift register (LFSR) pattern generators, the 16×16 MAC with a 32-bit accumulator as the circuit under test, a 32-bit multiple-input signature register (MISR) response analyzer, and a finite-state-machine controller that sequences initialization, pattern application, flushing, and comparison against the golden signature `0x1F5FAF38`. Three variants were synthesized under identical constraints: a baseline BIST implementation, a clock-gated variant, and an operand-isolated variant. A bus-invert-coded variant and a combined variant were also built but are excluded from the primary comparison because they increased total power; this negative result is reported for completeness. Stuck-at fault injection on the baseline gate-level netlist covered 980 net sites (1960 faults) over a 4096-pattern run and converged to 99.898% fault coverage, with two undetected faults on the MAC control inputs. Relative to the baseline, clock gating reduced total power by 4.7% with a 7.6% lower cell count and 4.2% lower area, whereas operand isolation reduced total power by 5.7% at a 1.5% area cost. A sixth variant, V6, combining operand isolation and clock gating, extends the comparison to six variants and reduced total power by 10.1% with a 4.9% lower cell count and 2.7% lower area, the largest power reduction of the variants compared. The undetected control-input faults, the area overhead of the BIST hardware, and the scope of the fault model are discussed as limitations.**

Index Terms*—Built-in self-test (BIST), linear feedback shift register (LFSR), multiple-input signature register (MISR), stuck-at fault model, fault coverage, clock gating, operand isolation, low-power design, multiply-accumulate (MAC), design for testability (DFT).

I. INTRODUCTION

Design-for-testability (DFT) is a standard requirement for digital arithmetic blocks that are embedded deep in a larger system-on-chip, where external automatic test equipment (ATE) access to internal nets is limited. Built-in self-test (BIST) addresses this by integrating pattern generation and response compaction on-chip, trading a modest area and routing overhead for at-speed, in-system testability [1], [2]. Multiply-accumulate (MAC) units are common targets for BIST because they are regular, arithmetic-dominated structures that appear repeatedly in DSP and machine-learning datapaths, and a stuck-at fault in the multiplier or accumulator logic can silently corrupt downstream computation.

This paper documents a complete, self-contained BIST implementation for a 16-bit unsigned MAC unit: a pseudo-random pattern generator built from two Fibonacci LFSRs, the MAC core as the circuit under test (CUT), a Galois-form MISR as the output response analyzer (ORA), and a controller finite-state machine (FSM) that sequences the test. Beyond basic testability, the paper evaluates two low-power DFT-compatible optimizations—clock gating and operand isolation—and reports their measured effect on post-synthesis power, area, and cell count relative to an un-optimized BIST baseline. A third technique, bus-invert coding, was also implemented and synthesized but is excluded from the headline comparison because it increased total power in the measured configuration; this negative result is reported for completeness rather than discarded. A sixth variant (V6), which combines operand isolation and clock gating, extends the study to six variants and is reported in Table IV with the same metrics as the other variants.

The contribution of this work is not a new BIST algorithm—LFSR/MISR-based self-test is well established—but a complete, simulation-verified case study in which the BIST architecture, the gate-level fault-injection results, and the post-synthesis power, area, and cell-count figures of the same design are reported together. The work additionally extends the low-power comparison with a combined clock-gating and operand-isolation variant (V6), with its power, area, and cell-count results.

II. RELATED WORK

LFSR-based pseudo-random pattern generation and MISR-based signature analysis were established as the standard BIST scheme in the STUMPS and related architectures, where feedback shift registers provide near-exhaustive, low-overhead stimulus and response compaction without requiring a stored test set [1]. The use of maximal-length LFSR polynomials to maximize pattern diversity, and of primitive MISR polynomials to minimize aliasing probability, is a well-known trade-off in signature-analysis testing [2], [3].

Low-power test is a parallel and increasingly important concern, since BIST pattern activity on arithmetic cores can exceed functional-mode switching activity and dominate peak and average power during test [4], [10]. Clock gating is a standard RTL and synthesis-level technique for suppressing unnecessary flip-flop toggling when a register's enable condition is false, and is widely supported by synthesis tools as automatic or RTL-inferred integrated clock gating (ICG) cells [5]. Operand isolation applies the same idea to combinational datapath inputs, blocking propagation of switching activity into unused arithmetic units such as multipliers, which is particularly effective for MAC-type structures with wide

combinational fan-in [6]. Bus-invert coding reduces the number of transitions on a bus by conditionally inverting the transmitted word, but its benefit depends on bus length, switching statistics, and the capacitance of the physical interconnect, and for short, on-chip buses with low intrinsic transition activity the added encoder/decoder logic can outweigh the savings [7]; this is consistent with the bus-inversion result observed in the present design and discussed in Section VII. Test-data and response compaction beyond a single MISR, such as test-vector compression [8] and X-compact response compaction [9], address orthogonal problems and are outside the scope of this work.

The present work differs from idealized treatments of these techniques by reporting them together, on the same RTL, under the same synthesis constraints, with the fault-coverage and power results obtained from the same project rather than drawn from separate publications.

III. PROPOSED BIST ARCHITECTURE

The BIST core, shown in Fig. 1, consists of four blocks that are identical across all synthesized variants: two LFSR-based test-pattern generators, the `mac16` CUT, a 32-bit MISR ORA, and a `bist_ctrl` FSM. A top-level wrapper (`bist_mac_top`) multiplexes test-mode and functional-mode operand sources into the MAC and instantiates the optional low-power structures (clock gating, operand isolation, bus-invert coding) around this common core, so that all variants are functionally and structurally identical apart from the added low-power hardware.

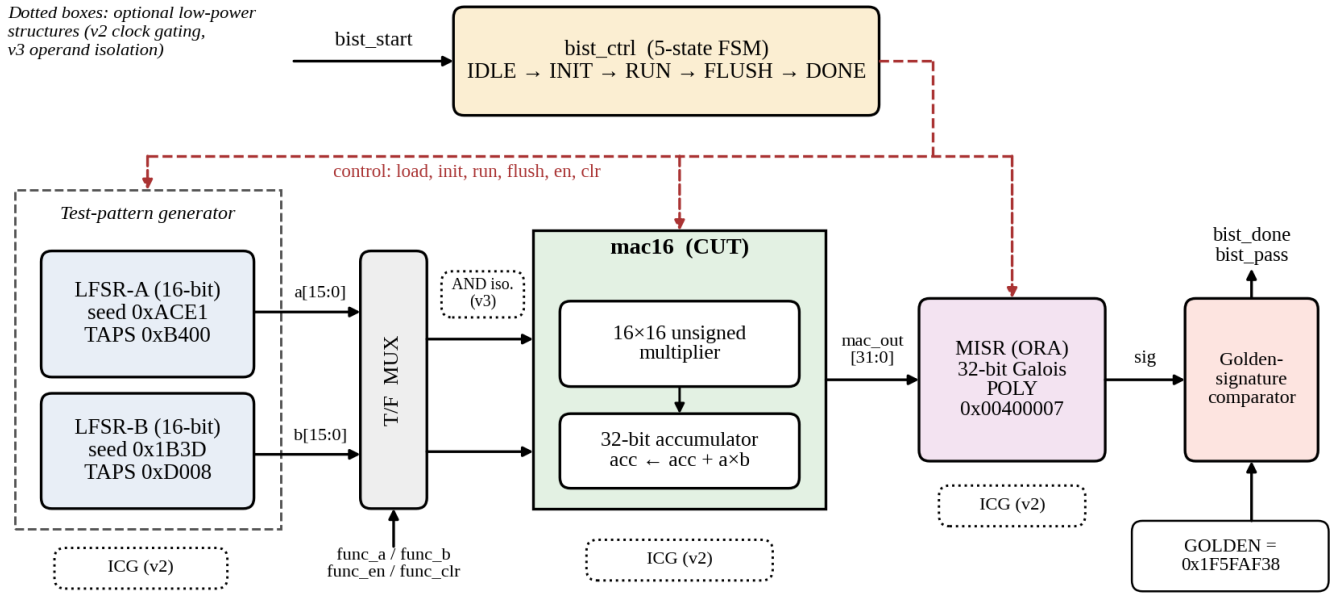


Fig. 1. Block diagram of the proposed BIST-enabled MAC: two LFSR pattern generators, the test/functional operand multiplexer, the `mac16` circuit under test (CUT), the 32-bit MISR output response analyzer, the golden-signature comparator, and the `bist_ctrl` FSM. Dotted blocks are the optional low-power structures of Section IV. The diagram is derived from the RTL hierarchy described in this section; it is not a synthesized schematic.

A. BIST Controller

The controller is a five-state FSM: IDLE → INIT → RUN (N cycles) → FLUSH → DONE → IDLE. On `bist_start`, INIT loads both LFSR seeds and clears the accumulator and MISR in one cycle; RUN applies N test patterns (N = 4096 in the synthesized configuration, set by the `NUM_PATTERNS` parameter and macro-overridable in simulation); FLUSH performs one additional MISR update to capture the final accumulator value; DONE asserts `bist_done` and `bist_pass`, the latter formed by comparing the captured signature against a GOLDEN parameter. The controller returns to IDLE only after `bist_start` is deasserted, so a single self-test run corresponds to one assertion of `bist_start`. The state sequence is illustrated in Fig. 2.

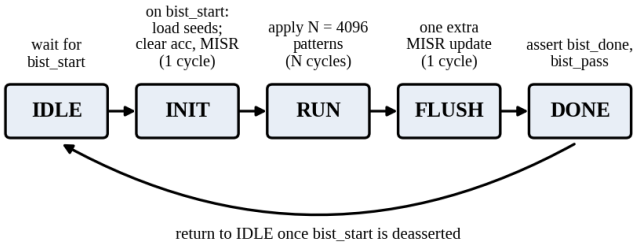


Fig. 2. State sequence of the `bist_ctrl` controller for one self-test run (N = 4096).

B. LFSR Test-Pattern Generators

Both operand generators are 16-bit Fibonacci-type LFSRs with external XOR feedback into bit 0 and a left-shift convention. Denoting the register state by q and the fixed tap vector by TAPS, the next state is

$$fb = \bigoplus(q \wedge \mathrm{TAPS}), \quad q^{+} = \{q[14:0], fb\} \tag{1}$$

where $\bigoplus(\cdot)$ denotes XOR-reduction (parity) of the masked register. The feedback bit is computed once per cycle from the full tap mask rather than from a chain of discrete XOR gates.

Operand-A generator (`u_lfsr_a`): seed `0xACE1`, TAPS = `0xB400`, corresponding to the characteristic polynomial

$$P_A(x) = x^{16} + x^{14} + x^{13} + x^{11} + 1 \quad (2)$$

Operand-B generator (`u_lfsr_b`): seed `0x1B3D`, TAPS = `0xD008`, corresponding to the characteristic polynomial

$$P_B(x) = x^{16} + x^{15} + x^{13} + x^4 + 1 \quad (3)$$

Both tap polynomials are maximal-length 16-bit LFSR polynomials, and this was checked with the project's bit-exact Python golden-reference model (`model/bist_model.py`): stepping each recurrence through its state sequence returns to the seed after $2^{16} - 1 = 65535$ states for both generators. Both LFSRs advance synchronously during INIT and RUN (`load` has priority over `en` for seed reload) and are held in reset to their seed values whenever `rst_n` is deasserted.

C. MAC Core (Circuit Under Test)

The CUT, `mac16`, is a 16×16 unsigned multiplier feeding a 32-bit accumulator: prod = a × b (32 bits, no saturation), acc ← acc + prod on every enabled cycle, with synchronous clear (clr, priority over en) and asynchronous active-low reset. In BIST mode the controller drives en during RUN and clr during INIT; in functional mode (BIST idle) the external `func_en`/`func_clr` signals drive the same ports through the top-level multiplexer.

D. MISR Output Response Analyzer

The ORA is a 32-bit Galois-form MISR with POLY = `0x00400007`. With din = `mac_out`, the next signature is

$$S(sig) = \{sig[30:0], 1'b0\} \bigoplus (sig[31] \cdot POLY) \quad (4)$$

$$sig^{+} = S(sig) \bigoplus din \quad (5)$$

where din is the 32-bit accumulator value. POLY corresponds to the primitive characteristic polynomial

$$P_M(x) = x^{32} + x^{22} + x^2 + x + 1 \quad (6)$$

The MISR is cleared synchronously to 0 at INIT (clr tied to the controller's init signal) and is updated once per cycle during RUN and once more during FLUSH, compacting the 32-bit accumulator value (din = `mac_out`) into a single running signature. This is the standard single-input-per-cycle MISR configuration applied to a wide (32-bit) response word rather than a true multi-input signature register sampling several independent outputs in parallel; the module name (`misr32`) reflects the response width, not multiple independent input streams.

E. Golden Signature

For N = 4096 applied patterns, the fault-free signature is GOLDEN = `0x1F5FAF38` (binary 0001 1111 0101 1111 1010 1111 0011 1000). It is set as the default GOLDEN parameter in `bist_mac_top.v` and is printed by the fault-simulation harness at the start of each campaign ("golden=1F5FAF38"). The value is computed by the bit-exact Python reference model (`model/bist_model.py`), which replicates the INIT/RUN/FLUSH timeline of the RTL cycle for cycle and matches the signature produced by the RTL; `bist_pass` is formed by comparing the captured signature with this value. The same golden value is the pass/fail reference for every fault-injection run reported in Section VII, since all runs used `NUM_PATTERNS` = 4096.

TABLE I
BIST CONFIGURATION PARAMETERS

Parameter	Value
Operand / accumulator width	16 bit / 32 bit
LFSR-A (`u_lfsr_a`)	seed `0xACE1`, TAPS `0xB400`
LFSR-B (`u_lfsr_b`)	seed `0x1B3D`, TAPS `0xD008`
MISR (`misr32`)	32 bit, Galois, POLY `0x00400007`
Pattern length N	4096 (`NUM_PATTERNS`)
Golden signature	`0x1F5FAF38`
Controller	5-state FSM (IDLE, INIT, RUN, FLUSH, DONE)

IV. LOW-POWER OPTIMIZATION TECHNIQUES

Three low-power techniques were implemented as optional hardware around the common BIST core, enabled by synthesis-time parameters (`CG_EN`, `OI_EN`, `BI_EN`) so that the BIST core itself is bit-identical across variants. A sixth variant, V6, combines the clock-gating and operand-isolation structures of Sections IV-A and IV-B and is described in Section IV-D.

A. Clock Gating (v2_clock_gating)

Each of the three clock domains—LFSR, MAC, MISR—is gated by a latch-based integrated clock-gating (ICG) cell: a transparent-while-clk-low enable latch feeding an AND with the free-running clock, gclk = clk & latched(en). This is the conventional glitch-free ICG structure that avoids combinational hazards on the enable reaching the AND gate. The enable conditions are `lfsr_cen` = init | run, `mac_cen` = `mac_en` | `mac_clr`, and `misr_cen` = init | run | flush, so each gated clock toggles only during the cycles in which its register must actually update; when `CG_EN` is active the corresponding per-flop enable is tied high (the clock itself performs the gating), consistent with standard practice of moving the gating from the data path to the clock tree to save clock-network power as well as register power.

B. Operand Isolation (v3_operand_isolation)

Operand isolation (`operand_iso`) is a purely combinational AND-mask on each 16-bit operand bus feeding the multiplier: dout = din & {W{en}}, with en tied to `mac_en` (the MAC's effective accumulate-enable, high during RUN and during functional accumulation). When the MAC is not enabled, both multiplier inputs are forced to zero, which prevents upstream LFSR or functional-operand toggling from propagating into the 16×16 multiplier tree and causing unnecessary combinational switching while the MAC's output is not being captured.

C. Bus-Invert Coding (v4_bus_inversion)—Excluded from Main Comparison

Bus-invert encoder/decoder pairs (`bus_invert_enc`/`bus_invert_dec`) were implemented on both 16-bit operand buses and synthesized as `v4_bus_inversion`, together with a combined variant (`v5_combined`) that includes all three techniques. Both results are listed in Table IV but are excluded from the primary comparison in Section VII, because bus-invert coding increased total power relative to the baseline (+48.1% for v4, +37.9% for v5). This agrees with the known limitation of bus-invert coding: its benefit depends on the transition

activity and physical capacitance of the coded bus, and on a short, low-activity, on-chip 16-bit operand bus the encoder/decoder and inversion-flag overhead can exceed the switching power it removes [7]. The result is therefore reported as a measured negative result.

D. Combined Operand Isolation and Clock Gating (V6)

The sixth variant, V6, enables clock gating and operand isolation together around the same common BIST core. Unlike `v5_combined`, it does not include bus-invert coding. The two techniques act on different parts of the design: the ICG cells of Section IV-A suppress clock transitions at the LFSR, MAC, and MISR registers when their enable conditions are false, whereas the AND-mask of Section IV-B stops operand toggling from propagating into the 16×16 multiplier when the MAC is disabled. Combining them therefore targets both sequential, clock-related activity and combinational datapath activity.

The post-synthesis results for V6 are reported in Section VII-B. The equivalence check of Section IV-E and the fault campaign of Section V are not reported for V6 and should not be assumed to transfer to it.

E. Functional Mode and Equivalence

Outside of a BIST run, the top-level wrapper routes `func_a`/`func_b`/`func_en`/`func_clr` directly to the (optionally isolated or bus-inverted) MAC datapath, so the same hardware serves both test and mission modes. An equivalence-checking testbench (`tb_equiv_check.v`) and a regression script (`run_regression.sh`) compare `mac_out` and the BIST signature of the five original variants (v1–v5) cycle by cycle against the baseline, in functional mode, BIST mode, idle phases, and with a reset applied in the middle of a run. All five of these variants matched the baseline in this check (0 mismatches over 13,941 compared cycles), so the low-power logic does not change the BIST behavior. Functional equivalence does not by itself imply identical fault coverage across variants, since the added low-power or bus-invert logic changes the gate-level netlist and therefore the fault universe; the fault campaign was run only on the baseline (`v1_normal`) netlist, and its coverage figures are not assumed to transfer unchanged to the other variants, including V6.

V. FAULT-INJECTION AND VERIFICATION METHODOLOGY

Fault coverage is evaluated by simulation-based stuck-at fault injection directly on the Genus-synthesized gate-level netlist of the baseline variant (`v1_normal`), not on the RTL and not through a commercial ATPG/fault-simulation tool such as Mentor Tessent or Synopsys TetraMAX. The method evaluates the detectability of stuck-at faults by the BIST pattern/response pair under event-driven gate-level simulation, which is suitable for coverage estimation but is not equivalent to an industrial ATPG signoff flow; it does not produce ATPG-compatible test patterns, fault dictionaries, or diagnosis data.

A. Fault Model and Site Enumeration

Fault sites are enumerated automatically from the gate-level netlist by parsing every input/output/inout/wire declaration inside the `mac16` module instance (the CUT), excluding `clk` and `rst_n`. Each eligible net (scalar or each bit of a vector) is assigned two fault instances, stuck-at-0 and stuck-at-1. For the baseline netlist this gives 980 distinct fault sites and 1960 stuck-at fault instances, as reported by the fault-simulation harness ("fault sites (nets): 980, total stuck-at faults: 1960"). No fault collapsing (equivalence or dominance) is applied; the 1960 figure is the raw site × polarity count. Only net-stem (whole-net) faults are modeled, so a stem with several fanout branches is represented once rather than once per branch; this is listed as a limitation in Section VIII.

B. Injection and Detection Procedure

For each of the 1960 faults, the gate-level testbench (`tb_fault_sim.v`) resets the design, forces the targeted net to its stuck-at value, runs the full BIST sequence for the configured `NUM_PATTERNS` (4096), and releases the force. A fault is scored as detected if the resulting MISR signature differs from the golden fault-free signature (`0x1F5FAF38`) at the corresponding pattern count. This is the standard signature-comparison criterion for MISR-based BIST; it carries a small probability of aliasing (a faulty response that produces the fault-free signature), which is inherent to any linear compaction scheme and is not separately quantified here. The signature is also sampled at intermediate pattern counts (32, 64, 128, 256, 512, 1024, 2048) during the same run, so a single 1960-fault campaign gives the full coverage-versus-pattern-length curve of Section VII without repeating the campaign at each length. The campaign is split into independent chunks (`FAULT_CHUNKS`, 8 in the reported run) for simulation throughput, which does not change the fault universe or the detection criterion. A short trial run (`FAULT_END=200`, faults 0–200 in 8 chunks) was used first to check the harness; it is not part of the reported coverage, which comes from the full 1960-fault campaign.

C. Coverage Calculation

Fault coverage (%) is calculated as:

$$\text{Coverage (\%)} = (\text{Detected} / \text{Total}) \times 100 \tag{7}$$

with "total modeled faults" taken as the full 1960-fault, 980-site universe described above, and "detected faults" taken from the full-campaign MISR comparison at each reported pattern count.

VI. SYNTHESIS AND EXPERIMENTAL SETUP

All variants were synthesized with Cadence Genus (Genus™ Synthesis Solution 21.1) under a single shared constraint set (`mac_bist.sdc`) and a single shared effort profile, so that differences in the reported PPA numbers reflect the added low-power/bus-invert logic rather than differing synthesis settings. The clock is constrained to a 10.0 ns period (100 MHz) with 0.10 ns clock uncertainty; primary inputs and outputs are given 2.0 ns input/output delay relative to clk; and `rst_n` is excluded from timing as a false path, consistent with its use as an asynchronous reset. The six variants (`v1_normal` ... `v5_combined`, and V6) share identical port lists and were synthesized with the same generic/map/optimization effort (medium) and the same RTL source list apart from the per-variant top-level wrapper.

TABLE II
SYNTHESIS AND CONSTRAINT SETUP

Item	Setting
Tool	Cadence Genus 21.1
Clock	10.0 ns (100 MHz), 0.10 ns uncertainty
I/O delay	2.0 ns input and output, relative to clk
`rst_n`	false path (asynchronous reset)
Effort	medium (generic / map / optimization)
Library file	`slow_vdd1v0.lib` (slow corner, 1.0 V)
Variants	`v1_normal` … `v5_combined` (same ports); V6 added

The standard-cell library is selected through `LIB_SEARCH_PATH` and `LIB_FILES` in `synthesis/scripts/config.tcl`; the library file used is `slow_vdd1v0.lib`, a slow-corner library with a 1.0 V nominal supply. Timing results (slack and critical-path delay) were not extracted for the variants, so the comparison in this paper is limited to cell count, area, and power, and no timing-closure claim is made.

Power was obtained from Genus after synthesis. The project flow also provides a switching-activity testbench (`tb_power_activity.v`, run through `sim/run_power_vcd.sh`) that applies identical stimulus to every variant: functional operation, idle phases with toggling inputs, a BIST run, and a final idle phase. The reported power values are post-synthesis estimates and therefore depend on the switching activity assumed for the workload. Fig. 3 shows the Genus Design Browser and schematic view of a pipelined MAC hierarchy (`mac_pipelined`, 819 leaf cells, 977 nets) synthesized in the same Genus 21.1 environment. It has 68 terminals (`a[15:0]`, `b[15:0]`, `clk`, `rst_n`, `en`, `clr`, and `mac_out[31:0]`) and no BIST interface ports, i.e., a pipelined MAC datapath without the BIST wrapper, so it is shown only as a reference for the synthesis environment and is not part of the PPA comparison in Table IV.

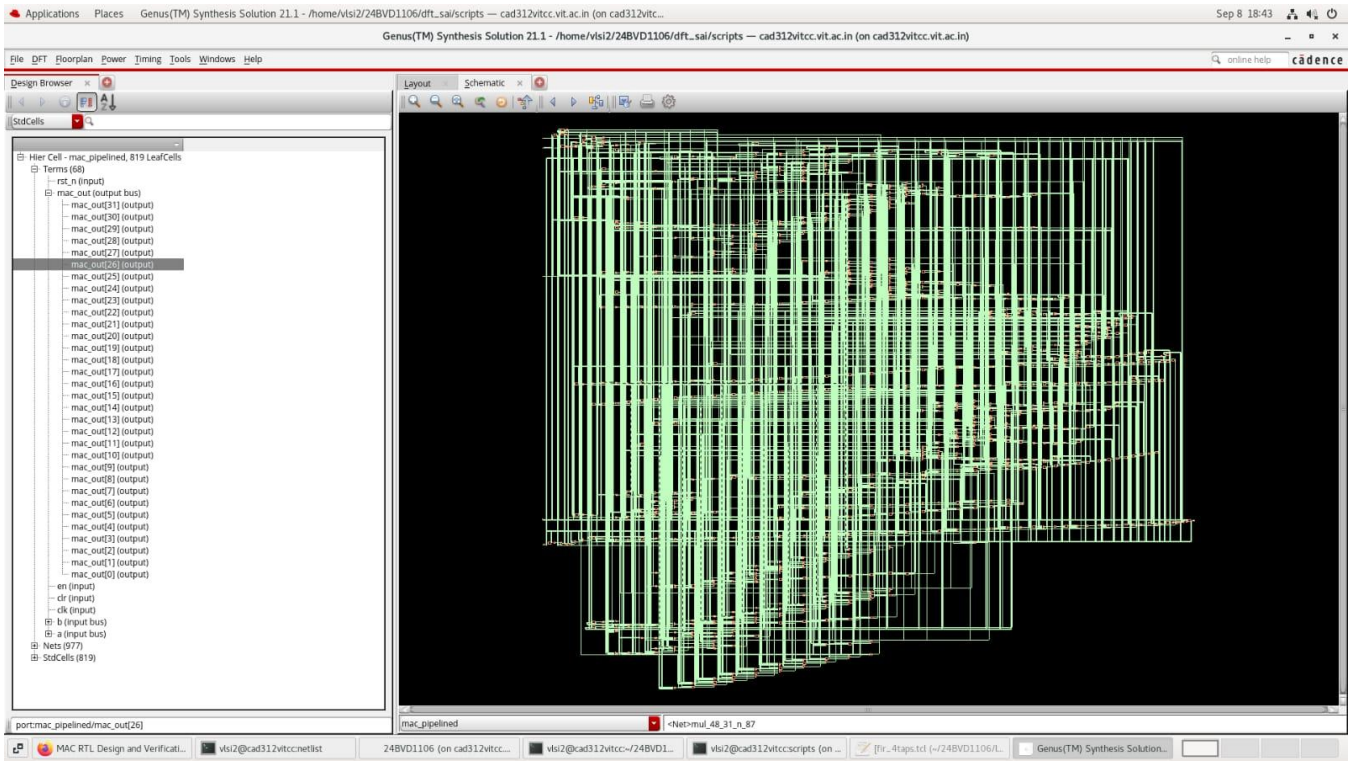


Fig. 3. Genus Design Browser and schematic view of the synthesized `mac_pipelined` hierarchy (819 leaf cells, 977 nets, 68 terminals: `a[15:0]`, `b[15:0]`, `clk`, `rst_n`, `en`, `clr`, and `mac_out[31:0]`). This is a pipelined MAC without the BIST wrapper, shown as a reference for the synthesis environment; it is not one of the variants in Table IV.

VII. RESULTS AND DISCUSSION

A. Fault Coverage versus Pattern Length

Table III reports the full fault campaign on the baseline (`v1_normal`) gate-level netlist: 1960 total modeled faults (980 sites × 2 polarities), golden signature `0x1F5FAF38`, with the signature sampled at the pattern counts below and at the final 4096-pattern flush.

TABLE III
FAULT COVERAGE VERSUS APPLIED PATTERN COUNT (BASELINE V1_NORMAL, 1960 FAULTS)

Patterns	Detected	Total	Coverage (%)
32	1952	1960	99.5918
64	1957	1960	99.8469
128	1958	1960	99.8980
256	1958	1960	99.8980
512	1958	1960	99.8980
1024	1958	1960	99.8980
2048	1958	1960	99.8980
4096 (final)	1958	1960	99.8980

Golden signature `0x1F5FAF38`; full 1960-fault campaign.

Coverage rises sharply over the first 64 patterns (99.59% → 99.85%, Fig. 4) as the pseudo-random LFSR sequence excites the faults and the MISR captures their effects, reaches its asymptotic value of 99.898% by 128 patterns, and is flat from 128 through the full 4096-pattern run. This plateau shows that the two faults still undetected at 128 patterns (Section VII-C) are not merely slow to excite—applying 32× more patterns does not detect them—so the cause is not an under-length test but the way the test schedule drives the MAC control inputs. The fault campaign was run on the baseline netlist only, so coverage is reported for `v1_normal`; the clock-gated and operand-isolated netlists differ structurally from the baseline, and their coverage is not assumed to be identical.

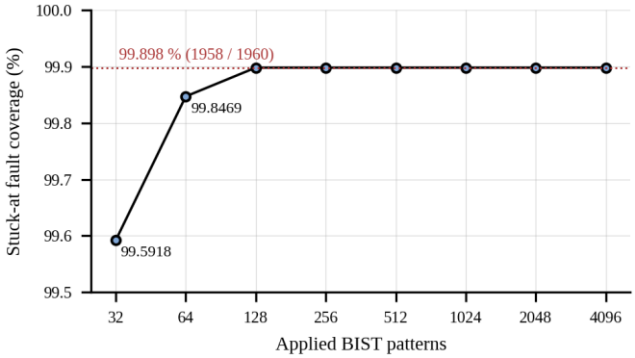


Fig. 4. Stuck-at fault coverage versus applied BIST patterns for the baseline (`v1_normal`) netlist, 1960 modeled faults (data of Table III). The vertical axis is truncated to 99.5–100%.

B. Power, Area, and Cell-Count Comparison

Table IV compares cell count, area, leakage power, internal power, switching power, and total power for all six variants, taken from the Genus reports (collected in `results/ppa_comparison.csv`). The primary comparison (Fig. 5) is restricted to `v1_normal` (baseline), `v2_clock_gating`, and `v3_operand_isolation`, and V6 (operand isolation + clock gating); `v4_bus_inversion` and `v5_combined` are listed in the same table for completeness but are excluded from the headline comparison because both increased total power relative to the baseline.

TABLE IV
POST-SYNTHESIS Power and Area COMPARISON OF ALL SIX VARIANTS

Variant	Cells	Δ Cells	Area (µm²)	Δ Area	Leak. (µW)	Int. (µW)	Sw. (µW)	Total (µW)	Δ Power
`v1_normal` (base)	1185	+0.0%	9926.74	+0.0%	42.732	612.542	229.083	884.357	+0.0%
`v2_clock_gating`	1095	−7.6%	9508.18	−4.2%	41.658	581.265	219.826	842.749	−4.7%
`v3_operand_iso.`	1217	+2.7%	10078.88	+1.5%	43.681	583.573	206.999	834.253	−5.7%
*`v4_bus_inversion`**	*1516*	*+27.9%*	*12354.88*	*+24.5%*	*56.118*	*872.138*	*381.808*	*1310.060*	*+48.1%*
*`v5_combined`**	*1466*	*+23.7%*	*11979.46*	*+20.7%*	*55.524*	*813.638*	*349.995*	*1219.160*	*+37.9%*
V6 (OI + CG)	1127	−4.9%	9660.32	−2.7%	42.583	553.775	198.634	794.993	−10.1%

Values from the Genus reports (`ppa_comparison.csv`); percentages are relative to `v1_normal`. Δ Cells is derived from the reported cell counts. *v4 and v5 are excluded from the primary comparison because they increased power. V6 = operand isolation + clock gating (OI + CG). Area is in µm².

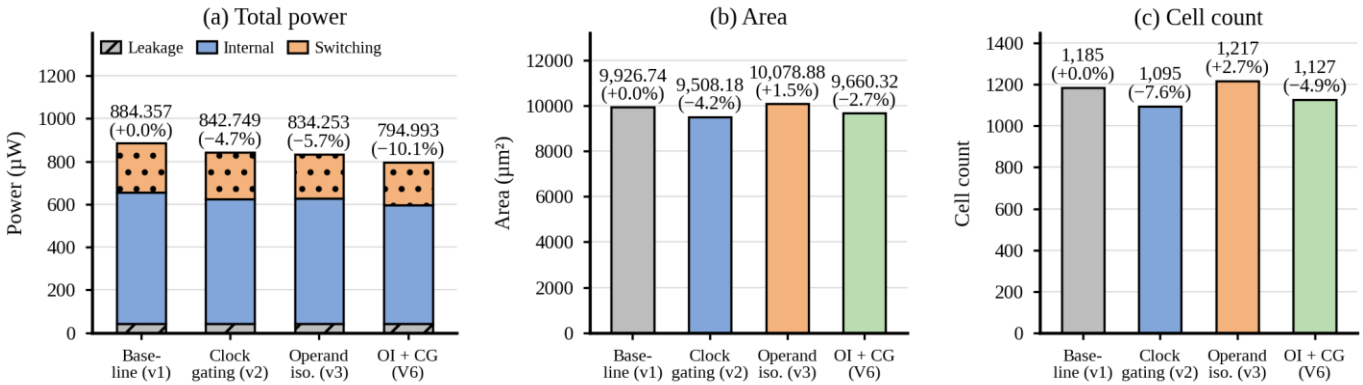


Fig. 5. Post-synthesis comparison of the baseline, clock-gated, operand-isolated, and combined operand-isolation and clock-gating (V6) variants: (a) total power split into leakage, internal, and switching components (µW), (b) area (µm²), and (c) cell count. Percentages are relative to the baseline; plotted values are those of Table IV.

Clock gating (v2) reduced cell count by 7.6% and area by 4.2%, and reduced total power by 4.7% relative to the baseline, with the reduction spread over all three power components (leakage −2.5%, internal −5.1%, switching −4.0%). The leakage reduction follows the smaller synthesized cell count rather than clock gating's main mechanism, which is the suppression of dynamic switching. Operand isolation (v3) increased cell count by 2.7% and area by 1.5%, consistent with

the added AND-isolation gates on both 16-bit operand buses, yet gave the largest total-power reduction of the original three compared variants (v1–v3) (−5.7%). This is driven mainly by a 9.6% reduction in switching power, which matches the intended effect of blocking multiplier-input toggling while the MAC is idle. Internal power for v3 is also lower than the baseline (−4.7%) despite the higher cell count, while its leakage power is slightly higher (+2.2%) because of the added isolation gates.

The sixth variant, V6, combines the two optimizations. Clock gating suppresses unnecessary clock transitions in the relevant sequential blocks, while operand isolation prevents unnecessary switching from propagating into the multiplier when the MAC is disabled, so the combination targets both sequential clock-related activity and combinational datapath activity. V6 gave the lowest total power of all variants, 794.993 µW, which is 10.1% below the 884.357 µW baseline and lower than clock gating alone (842.749 µW, −5.7%) and operand isolation alone (834.253 µW, −4.7%). The reduction is spread over all three components relative to the baseline (leakage −0.3%, internal −9.6%, switching −13.3%), with the largest relative saving in switching power. The individual total-power reductions of clock gating (41.608 µW) and operand isolation (50.104 µW) sum to 91.712 µW, close to the 89.364 µW reduction of V6, which is consistent with the two techniques acting largely on complementary sources of switching activity.

V6 reduced cell count by 4.9% (1127 cells) and area by 2.7% (9660.32 µm²) relative to the baseline. Its cell count and area are higher than those of clock gating alone (+2.9% and +1.6%) because V6 also carries the AND-isolation gates, but lower than those of operand isolation alone (−7.4% and −4.2%), so the area overhead of operand isolation is more than offset by the cell reduction that accompanies clock gating in this design. Implementation complexity is higher than for either technique alone, since the clock-gating enables and the isolation enable must be derived consistently from the controller and functional-mode signals, and the gated-clock and isolation paths must be verified together for functional equivalence and timing, neither of which is reported for V6 here. Because V6 changes the gate-level netlist, the fault-coverage results of Section VII-A are likewise not assumed to apply to it.

Both excluded variants show increases across every component: v4 (bus inversion) increases total power by 48.1% and v5 (combined) by 37.9%, with cell count and area increasing by about 24–28% and 21–25% respectively. As discussed in Section IV-C, this is attributed to the encoder/decoder and inversion-flag overhead of the bus-invert coding exceeding its switching-power savings on short, on-chip, 16-bit operand buses; a quantitative breakdown would need the per-bus transition statistics, which were not extracted in this work.

C. Undetected Faults

Of the 1960 modeled faults in the full baseline campaign, 2 remained undetected after the full 4096-pattern run (1958 of 1960 detected, 99.898% coverage), matching the difference between the total fault count and the final detected count in Table III. The undetected-fault log (`undetected_v1_normal.txt`) lists these as shown in Table V.

TABLE V
UNDETECTED FAULTS, FULL BASELINE CAMPAIGN, 4096 PATTERNS

Fault ID	Polarity	Net	Note
1	SA1 (stuck-at-1)	en	MAC accumulate-enable input
2	SA0 (stuck-at-0)	clr	MAC synchronous clear input

Both undetected faults are on the `mac16` control inputs `en` and `clr` rather than on the datapath (operand, product, or accumulator bits). Both are masked by the way the BIST controller drives these inputs. During RUN the controller holds `mac_en` high and `mac_clr` low for the whole pattern-application window, so a stuck-at-1 `en` changes nothing there; in INIT, `clr` has priority over `en`, and in FLUSH the MISR samples the accumulator before the extra accumulation that a stuck-high `en` would cause, so the signature is unchanged. For `clr` stuck-at-0, the accumulator is already zero because the asynchronous reset is applied before each run, so the clear in INIT has no effect on the starting state. Detecting these two faults requires a test step that exercises the control inputs, such as deasserting `en` and pulsing `clr` during the run, or repeating the BIST without an intervening reset. No other undetected faults were found on the operand, product, or accumulator nets.

VIII.. CONCLUSION

A complete LFSR/MISR BIST architecture for a 16-bit MAC was implemented, synthesized in three primary variants (baseline, clock-gated, operand-isolated), and verified by simulation-based stuck-at fault injection on the baseline gate-level netlist. The fault campaign covered 1960 modeled faults (980 sites) over a 4096-pattern BIST run and converged to 99.898% coverage, with two undetected faults, both on the `en` and `clr` control inputs. Relative to the baseline, clock gating reduced total power by 4.7% with a 7.6% lower cell count and a 4.2% lower area, while operand isolation achieved a larger 5.7% total-power reduction, driven mainly by reduced switching power, at a modest 1.5% area cost. A sixth variant combining operand isolation and clock gating (V6) extends the comparison to six variants and achieved the largest total-power reduction (10.1%, 794.993 µW) with a 4.9% lower cell count and 2.7% lower area than the baseline; its fault coverage and equivalence were not evaluated in this work. A bus-invert-coded variant was also built but increased power (+48.1%) on this design's short operand buses and was therefore excluded from the primary comparison. The paper documents the LFSR/MISR feedback equations, the golden signature, the fault model and its net-stem-only scope, and the measured PPA trade-offs, and it discusses the limitations of the approach, including the undetected control-input faults and the area overhead of the BIST hardware.

REFERENCES

[1] V. D. Agrawal, C. R. Kime, and K. K. Saluja, "A tutorial on built-in self-test, part 1: Principles," IEEE Design & Test of Computers, vol. 10, no. 1, pp. 73–82, 1993.

[2] C. R. Kime, V. D. Agrawal, and K. K. Saluja, "A tutorial on built-in self-test, part 2: Applications," IEEE Design & Test of Computers, vol. 10, no. 2, pp. 69–77, 1993.

[3] P. H. Bardell, W. H. McAnney, and J. Savir, Built-In Test for VLSI: Pseudorandom Techniques. New York, NY, USA: Wiley, 1987.

[4] S. Devadas and S. Malik, “A survey of optimization techniques targeting low power VLSI circuits,” in Proc. 32nd ACM/IEEE Design Automation Conf. (DAC), 1995, pp. 242–247.

[5] M. Alidina et al., “Precomputation-based sequential logic optimization for low power,” IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 2, no. 4, pp. 426–436, Dec. 1994.

[6] L. Benini, G. De Micheli, and E. Macii, “Designing low-power circuits: Practical recipes,” IEEE Circuits and Systems Magazine, vol. 1, no. 1, pp. 6–25, 2001.

[7] M. R. Stan and W. P. Burleson, “Bus-invert coding for low-power I/O,” IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 3, no. 1, pp. 49–58, Mar. 1995.

[8] N. A. Touba, “Survey of test vector compression techniques,” IEEE Design & Test of Computers, vol. 23, no. 4, pp. 294–303, 2006.

[9] S. Mitra and K. S. Kim, “X-Compact: An efficient response compaction technique,” IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems, vol. 23, no. 3, pp. 421–432, 2004.

[10] P. Girard, “Survey of low-power testing of VLSI circuits,” IEEE Design & Test of Computers, vol. 19, no. 3, pp. 80–90, 2002.